



TRƯỜNG ĐẠI HỌC SƯ PHẠM KỸ THUẬT VĨNH LONG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO TIỂU LUẬN
TỰ TRIỂN KHAI WEB SERVER NHỎ BẰNG DOCKER VÀ MÔ PHỎNG LỖ
HỒNG CẦU HÌNH

Sinh viên thực hiện

Nguyễn Văn Tân

22004329

Giáo viên hướng dẫn

ThS. Trần Thái Bảo

Vĩnh Long, 2026

NHIỆM VỤ TIỂU LUẬN AN TOÀN ĐIỆN TOÁN Đám Mây

Tên tiểu luận: Tự triển khai web server nhỏ bằng docker và mô phỏng lỗ hổng cấu hình

Nhiệm vụ:

- Cài đặt docker
- Deploy Nginx/PHP
- Cố tình cấu hình sai quyền truy cập sau đó phân tích lỗi và khắc phục

Phương pháp đánh giá: *Báo cáo trước hội đồng* *Chấm thuyết minh*

Ngày giao tiểu luận: *ngày tháng năm*

Ngày hoàn thành tiểu luận: *ngày tháng năm*

Sinh viên thực hiện tiểu luận:

Họ và tên sinh viên: Nguyễn Văn Tân

MSSV: 22004329

Vĩnh Long, ngày tháng năm

Trưởng Khoa/Bộ môn

Người hướng dẫn

(Ký và ghi rõ họ tên)

(Ký và ghi rõ họ tên)

NHẬN XÉT & ĐÁNH GIÁ ĐIỂM CỦA NGƯỜI HƯỚNG DẪN

Ý thức thực hiện:

.....

.....

.....

Nội dung thực hiện:

.....

.....

.....

Hình thức trình bày:

.....

.....

.....

Tổng hợp kết quả:

.....

.....

.....

- ☐ Tổ chức báo cáo trước hội đồng
- ☐ Tổ chức chấm thuyết minh

Vĩnh Long, ngày.....tháng.....năm.....

Người hướng dẫn

LỜI CẢM ƠN

Trước hết, nhóm chúng em xin cảm ơn sự dẫn dắt của thầy Trần Thái Bảo cũng như thầy Nguyễn Khắc Tường đã dìu dắt chúng em trên con đường hoàn thiện kiến thức và thể hiện năng lực của bản thân. Trong suốt quá trình học tập và thực hiện bài báo cáo, nhóm chúng em đã nhận được kiến thức để hiểu rõ hơn về các vấn đề triển khai hệ thống, an toàn thông tin và tư duy thiết kế phòng thủ trong môi trường thực tế.

Nhóm chúng em cũng xin cảm ơn thầy đã tạo điều kiện để chúng em được tiếp cận mô hình học tập gắn với thực hành, từ đó có cơ hội mô phỏng các tình huống cấu hình sai thường gặp trong triển khai Docker và rút ra bài học về cách đánh giá rủi ro, phân tích nguyên nhân cũng như xây dựng biện pháp khắc phục hiệu quả. Đây là trải nghiệm quan trọng giúp nhóm chúng em củng cố kiến thức nền tảng, đồng thời rèn luyện kỹ năng làm việc nhóm, tổ chức nội dung báo cáo và trình bày vấn đề một cách mạch lạc.

Dù đã cố gắng hoàn thiện, bài báo cáo chắc chắn vẫn còn những thiếu sót do hạn chế về thời gian và kinh nghiệm thực tế. Nhóm chúng em kính mong nhận được sự góp ý từ thầy cô để có thể tiếp tục cải thiện và hoàn thiện hơn trong những lần thực hiện sau. Một lần nữa, nhóm chúng em xin chân thành cảm ơn thầy cô.

Xin chân thành cảm ơn!

LỜI CẢM ƠN	4
CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI.....	7
1.1. Tính cấp thiết của đề tài.....	7
1.2. Mục tiêu nghiên cứu	7
1.3. Phạm vi thực hiện	8
1.4. Phương pháp nghiên cứu	8
CHƯƠNG 2. CƠ SỞ LÝ THUYẾT	10
2.1. Docker và Container	10
2.2. Kiến trúc Web Server	11
2.3. Docker Compose	13
2.4. Bảo mật hệ thống	14
CHƯƠNG 3. XÂY DỰNG HỆ THỐNG	16
3.1. Mô hình hệ thống	16
3.2. Chuẩn bị môi trường	16
3.3. Dockerfile	17
3.4. Docker Compose	18
3.5. Cấu hình Nginx	20
3.6. Triển khai và kiểm thử	20

CHƯƠNG 4. PHÂN TÍCH RỦI RO TỪ CÁC LỖ HỒNG CẤU HÌNH	21
4.1. Giới thiệu chương	21
4.2. Rủi ro từ việc công khai PostgreSQL ra mạng bên ngoài	21
4.3. Rủi ro từ Docker Socket Exposure	23
4.4. Rủi ro từ Container chạy quyền Root và Flat Network	25
CHƯƠNG 5. GIẢI PHÁP KHẮC PHỤC VÀ HARDENING HỆ THỐNG	28
5.1. Giới thiệu chương	28
5.2. Khắc phục PostgreSQL Exposure	28
5.3. Khắc phục Docker Socket Exposure	30
5.4. Áp dụng nguyên tắc Least Privilege	32
5.5. Phân tách mạng Docker	33
5.6. Hardening hệ thống	35
5.7. Đánh giá sau khi khắc phục	38
CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	40
6.1. Kết quả đạt được	40
6.2. Hạn chế của đề tài	41
6.3. Hướng phát triển	41
6.4. Bài học kinh nghiệm	42
TÀI LIỆU THAM KHẢO	44

CHƯƠNG 1. TỔNG QUAN ĐỀ TÀI

1.1. Tính cấp thiết của đề tài

Trong kỷ nguyên phát triển phần mềm hiện đại, công nghệ container hóa (Containerization) - đặc biệt là Docker - đã trở thành tiêu chuẩn cốt lõi để đóng gói và vận hành ứng dụng một cách nhanh chóng, nhất quán. Tuy nhiên, sự tiện lợi và tốc độ triển khai của Docker thường vô tình tạo ra một lỗ hổng lớn trong nhận thức bảo mật của các kỹ sư: việc tập trung quá mức vào tính năng hoạt động của ứng dụng mà lơ là công tác siết chặt cấu hình hệ thống.

Theo thống kê từ tổ chức bảo mật OWASP, lỗ hổng cấu hình sai bảo mật (Security Misconfiguration) luôn nằm trong nhóm các rủi ro nguy hiểm và phổ biến nhất. Các sai sót như phơi nhiễm cổng cơ sở dữ liệu ra ngoài môi trường mạng LAN, lạm dụng quyền hạn tối cao (root) trong container, hoặc để lộ tệp tin điều khiển hệ thống (docker.sock) có thể dẫn đến hậu quả nghiêm trọng: hacker chiếm quyền điều khiển toàn bộ máy chủ, đánh cắp dữ liệu hoặc phá hủy hệ thống. Do đó, việc nghiên cứu đề tài "Tự triển khai web server nhỏ bằng docker và mô phỏng lỗ hổng cấu hình" ngay trên môi trường thực nghiệm cục bộ là vô cùng cấp thiết, giúp sinh viên làm chủ quy trình quản trị và thiết lập các giải pháp phòng thủ (Hardening) thực tế.

1.2. Mục tiêu nghiên cứu

Đề tài được thực hiện nhằm đạt được các mục tiêu sau:

1.2.1. Mục tiêu tổng quát

Xây dựng và triển khai một hệ thống Web Server sử dụng Docker, đồng thời mô phỏng các lỗ hổng cấu hình phổ biến nhằm đánh giá mức độ ảnh hưởng và đề xuất giải pháp khắc phục phù hợp.

1.2.2. Mục tiêu cụ thể

- Tìm hiểu công nghệ Docker và mô hình container hóa ứng dụng.
- Xây dựng môi trường triển khai ứng dụng web sử dụng Docker Compose.
- Triển khai ứng dụng Laravel trên nền tảng PHP-FPM, Nginx và PostgreSQL.
- Tìm hiểu cơ chế hoạt động của mô hình Web Server nhiều tầng.

- Mô phỏng một số lỗ hổng cấu hình thường gặp trong quá trình triển khai hệ thống.
- Phân tích các nguy cơ bảo mật phát sinh từ các lỗi cấu hình.
- Đề xuất và thực hiện các biện pháp khắc phục nhằm nâng cao mức độ an toàn cho hệ thống.
- Đánh giá hiệu quả của các giải pháp bảo mật sau khi áp dụng.

1.3. Phạm vi thực hiện

Đề tài được thực hiện trong phạm vi xây dựng và triển khai một hệ thống Web Server quy mô nhỏ trên môi trường máy chủ cục bộ (Host) phục vụ mục đích học tập và nghiên cứu thực nghiệm. Các công nghệ cốt lõi được sử dụng bao gồm Docker Desktop, Docker Compose, Laravel Framework, PHP-FPM, Nginx Web Server và hệ quản trị cơ sở dữ liệu PostgreSQL.

Hệ thống vận hành theo mô hình kiến trúc ba tầng tiêu chuẩn: Nginx đóng vai trò Web Server và Reverse Proxy điều phối yêu cầu; PHP-FPM thực thi mã nguồn Laravel; và PostgreSQL lưu trữ dữ liệu ứng dụng. Đề tài tập trung nghiên cứu, giả lập và phân tích kỹ thuật các lỗ hổng cấu hình sai bảo mật (Security Misconfiguration) phổ biến trong môi trường ảo hóa bao gồm: lạm dụng quyền hạn tối cao (root), phơi nhiễm tệp tin điều khiển hệ thống (docker.sock) và phơi nhiễm dịch vụ cơ sở dữ liệu ra môi trường mạng nội bộ (LAN). Từ đó, đề tài áp dụng các giải pháp đóng gói an toàn (Hardening) để khắc phục triệt để. Các nội dung nằm ngoài hạ tầng container cục bộ như điện toán đám mây (Cloud Computing), dịch vụ điều phối Kubernetes, bộ cân bằng tải Load Balancer độc lập, hệ thống giám sát chuyên sâu hoặc chuỗi tự động hóa CI/CD không nằm trong phạm vi nghiên cứu của đề tài.

1.4. Phương pháp nghiên cứu

Để đạt được các mục tiêu đề ra, đề tài sử dụng kết hợp nhiều phương pháp nghiên cứu khác nhau.

1.4.1. Phương pháp nghiên cứu lý thuyết

Thu thập, phân tích và tổng hợp hệ thống tài liệu chính thống để xây dựng nền tảng kiến thức vững chắc phục vụ triển khai và bảo mật hệ thống:

- Tài liệu kỹ thuật cốt lõi: Docker Documentation, Docker Compose Specification, Laravel, Nginx và PostgreSQL Documentation.
- Tiêu chuẩn an toàn thông tin: OWASP Top 10 và bộ tài liệu hướng dẫn siết chặt bảo mật CIS Docker Benchmark.

1.4.2. Phương pháp thực nghiệm

Tiến hành xây dựng hệ thống Web Server thực tế bằng Docker Compose, bao gồm:

- Khởi tạo hạ tầng container cô lập bằng Docker Compose gồm: Container Nginx, Container PHP-FPM và Container PostgreSQL.
- Chủ động giả lập các kịch bản tấn công khai thác lỗi cấu hình sai (Misconfiguration) ngay trên môi trường Lab để quan sát hành vi và đánh giá phạm vi ảnh hưởng thực tế.

1.4.3. Phương pháp phân tích và đánh giá

Các kết quả thu được từ quá trình thực nghiệm được phân tích dựa trên các tiêu chí bảo mật cơ bản gồm:

- Tính bảo mật (Confidentiality).
- Tính toàn vẹn (Integrity).
- Tính sẵn sàng (Availability).

Đồng thời tiến hành so sánh trạng thái hệ thống trước và sau khi áp dụng các biện pháp khắc phục nhằm đánh giá hiệu quả của các giải pháp bảo mật được đề xuất.

CHƯƠNG 2. CƠ SỞ LÝ THUYẾT

2.1. Docker và Container

2.1.1. Container

Container là giải pháp ảo hóa ở cấp độ hệ điều hành, cho phép đóng gói toàn bộ mã nguồn ứng dụng, các thư viện phụ thuộc, tệp cấu hình và môi trường thực thi vào một đơn vị độc lập. Khác với máy ảo (Virtual Machine), container không mang theo hệ điều hành riêng mà chia sẻ chung nhân (Kernel) của máy chủ cục bộ (Host). Cơ chế này giúp tối ưu hóa hiệu năng, tiết kiệm bộ nhớ, giảm dung lượng lưu trữ và tăng tốc độ khởi động. Nhờ tính đồng bộ cao, container đảm bảo ứng dụng vận hành nhất quán trên mọi môi trường máy chủ thực nghiệm.

2.1.2. Docker

Docker là nền tảng mã nguồn mở hàng đầu hỗ trợ tự động hóa quy trình xây dựng, đóng gói và vận hành ứng dụng dưới dạng container. Docker cung cấp hệ sinh thái toàn diện bao gồm công cụ tạo lập ảnh đóng gói (Image), quản lý vòng đời container, thiết lập mạng nội bộ (Network) và phân vùng lưu trữ (Volume).

2.1.3. Docker Image

Docker Image là một khuôn mẫu chỉ đọc (Read-only template) chứa các chỉ thị cấu hình để khởi tạo Docker Container. Image bao gồm hệ điều hành nền, môi trường thực thi, các thư viện hệ thống và mã nguồn ứng dụng. Trong đề tài này, ảnh đóng gói của tầng ứng dụng được xây dựng từ một tệp chỉ thị (Dockerfile) dựa trên nền tảng FROM php:8.5-fpm. Quá trình biên dịch (Build) sẽ tạo ra một Image chứa trọn vẹn môi trường chạy cho framework Laravel.

2.1.4. Docker Container

Container là một thực thể hoạt động (Runtime instance) được khởi tạo từ Docker Image. Trong phạm vi nghiên cứu, hệ thống bao gồm ba container dịch vụ lõi vận hành độc lập:

Các container hoạt động độc lập nhưng có thể giao tiếp với nhau thông qua Docker Network.

2.1.5. Docker Network

Docker Network là cơ chế cung cấp khả năng kết nối và truyền thông nội bộ giữa các container với nhau, hoặc giữa container với môi trường mạng bên ngoài mà không cần phải công khai toàn bộ các cổng dịch vụ nhạy cảm ra môi trường Internet. Đề tài áp dụng kỹ thuật phân tách mạng bằng cách thiết lập hai vùng mạng riêng biệt:

Mạng frontend_net: Chỉ chứa container Nginx tiếp nhận các yêu cầu truy cập từ người dùng.

Mạng backend_net: Chứa container PHP-FPM và container PostgreSQL chỉ phục vụ giao tiếp nội bộ giữa ứng dụng và cơ sở dữ liệu.

Mô hình phân tách này giúp cô lập hoàn toàn cơ sở dữ liệu, ngăn chặn các nguy cơ truy cập trái phép từ bên ngoài vào vùng lõi dữ liệu.

2.1.6. Docker Volume

Docker Volume là cơ chế chính thức được Docker khuyến nghị nhằm phục vụ mục đích lưu trữ dữ liệu bền vững (persistent data) độc lập với vòng đời của container. Theo cơ chế mặc định, khi một container bị xóa bỏ, toàn bộ dữ liệu phát sinh bên trong lớp ghi của nó sẽ bị mất hoàn toàn. Để giải quyết vấn đề này, đề tài cấu hình một volume có tên postgres_data liên kết trực tiếp vào thư mục lưu trữ của hệ quản trị cơ sở dữ liệu PostgreSQL. Nhờ đó, toàn bộ dữ liệu nghiệp vụ của ứng dụng Laravel vẫn được toàn vẹn và giữ nguyên ngay cả khi các container bị khởi động lại hoặc phá hủy.

2.2. Kiến trúc Web Server

2.2.1. Tổng quan kiến trúc

Hệ thống được triển khai theo mô hình kiến trúc ứng dụng ba tầng tiêu chuẩn bao gồm: Máy chủ Web (Web Server), Máy chủ Ứng dụng (Application Server) và Máy chủ Cơ sở dữ liệu (Database Server). Luồng xử lý request được tổ chức theo sơ đồ tuyến tính:

Client -> Nginx -> PHP-FPM (Laravel) -> PostgreSQL

Trong cấu trúc này, mỗi thành phần được phân định trách nhiệm rõ ràng nhằm tăng cường hiệu năng xử lý hệ thống, tối ưu hóa công tác quản trị và tạo tiền đề cho việc mở rộng hạ tầng theo chiều ngang khi lưu lượng truy cập tăng cao.

2.2.2. Nginx

Nginx là một máy chủ web mã nguồn mở có hiệu năng cao, hoạt động dựa trên kiến trúc hướng sự kiện (event-driven) và bất đồng bộ. Trong phạm vi đề tài này, Nginx được định cấu hình tại tầng biên để đảm nhận các nhiệm vụ:

- Tiếp nhận và xử lý trực tiếp các yêu cầu HTTP từ phía client.
- Phục vụ các tài nguyên tĩnh của ứng dụng như hình ảnh, tệp tin CSS và JavaScript nhằm giảm tải cho tầng ứng dụng.
- Chuyển tiếp các yêu cầu xử lý logic (yêu cầu động) sang dịch vụ xử lý mã nguồn PHP-FPM.
- Đóng vai trò như một lá chắn bảo mật để ẩn hoàn toàn cấu trúc bên trong của PHP-FPM trước môi trường Internet công cộng.

Nginx sở hữu các ưu điểm vượt trội như tiêu thụ rất ít tài nguyên phần cứng, hoạt động ổn định cao và hỗ trợ cơ chế làm máy chủ ủy quyền ngược Reverse Proxy rất mạnh mẽ.

2.2.3. PHP-FPM

PHP-FPM là cụm từ viết tắt của FastCGI Process Manager, đây là trình quản lý tiến trình xử lý mã nguồn PHP được thiết kế nhằm tối ưu hóa hiệu năng cho các website có lưu lượng truy cập lớn. Nhiệm vụ cốt lõi của PHP-FPM trong hệ thống là tiếp nhận mã nguồn ứng dụng Laravel từ yêu cầu chuyển tiếp của Nginx thông qua giao thức FastCGI, thực thi các logic nghiệp vụ của chương trình, thực hiện các truy vấn dữ liệu và trả kết quả đã biên dịch về cho Web Server. Trong cấu trúc tệp tin Docker Compose của đề tài, dịch vụ mang tên app chính là một container vận hành tiến trình PHP-FPM này.

2.2.4. PostgreSQL

PostgreSQL là một hệ quản trị cơ sở dữ liệu quan hệ hướng đối tượng mã nguồn mở mạnh mẽ, nổi tiếng với độ ổn định cao, khả năng quản lý dữ liệu lớn đáng tin cậy và

tuân thủ nghiêm ngặt các tiêu chuẩn mã nguồn SQL. Trong đề tài này, PostgreSQL đóng vai trò làm kho lưu trữ dữ liệu tập trung cho ứng dụng thương mại điện tử Laravel, chịu trách nhiệm lưu trữ và quản lý các bảng thông tin nghiệp vụ quan trọng bao gồm danh mục tài khoản người dùng, thông tin sản phẩm, trạng thái đơn hàng và các bản ghi log hệ thống. Dịch vụ PostgreSQL được cấu hình bảo mật nghiêm ngặt và chỉ chấp nhận kết nối từ các nguồn được định danh trong mạng nội bộ.

2.2.5. Reverse Proxy

Reverse Proxy là mô hình kiến trúc máy chủ trung gian được đặt phía trước các máy chủ ứng dụng nội bộ. Khi có yêu cầu kết nối từ phía client, Reverse Proxy sẽ tiếp nhận yêu cầu đó tại tầng biên, sau đó điều hướng luồng dữ liệu đến các dịch vụ xử lý thích hợp ở phía sau. Trong đề tài, Nginx đóng vai trò là một Reverse Proxy nằm chắn giữa Client và mạng chứa container PHP-FPM. Luồng đi cụ thể:

Client -> Máy chủ Nginx (Reverse Proxy) -> Container PHP-FPM

Cơ chế này mang lại lợi ích lớn về mặt an toàn thông tin do người dùng ngoài Internet không thể nhận biết hoặc tương tác trực tiếp với máy chủ ứng dụng nội bộ, giúp ngăn chặn hiệu quả các cuộc tấn công khai thác dịch vụ trực tiếp, đồng thời hỗ trợ tốt cho việc triển khai mã hóa HTTPS tập trung tại một điểm biên duy nhất.

2.3. Docker Compose

2.3.1. Khái niệm

Docker Compose là một công cụ hỗ trợ đắc lực trong việc định nghĩa và vận hành các ứng dụng Docker phức tạp chạy đa container. Bằng cách sử dụng một tệp tin cấu hình duy nhất được định dạng theo chuẩn YAML với tên gọi mặc định là docker-compose.yml, người quản trị hệ thống có thể dễ dàng khai báo toàn bộ các thông số kỹ thuật của các dịch vụ, các phân vùng mạng nội bộ kết nối và các ổ đĩa lưu trữ dữ liệu đi kèm.

2.3.2. Vai trò trong đề tài

Trong quá trình triển khai hệ thống Web Server này, Docker Compose đóng vai trò trung tâm điều phối giúp tự động hóa hoàn toàn các thao tác hạ tầng bao gồm:

- Khởi tạo đồng thời ba dịch vụ độc lập: Nginx, PHP-FPM và PostgreSQL.
- Thiết lập và liên kết chính xác hai vùng mạng cô lập frontend_net và backend_net.
- Gắn kết ổ đĩa lưu trữ postgres_data cho cơ sở dữ liệu.

Thay vì phải gõ thủ công hàng loạt câu lệnh Docker CLI phức tạp với nguy cơ sai sót cao, người dùng chỉ cần thực thi một câu lệnh duy nhất:

```
docker compose up -d
```

Toàn bộ kiến trúc máy chủ ba tầng sẽ tự động được khởi dựng và đưa vào trạng thái sẵn sàng hoạt động trong một quy trình thống nhất.

2.3.3. Ưu điểm

- Quản lý tập trung.
- Dễ bảo trì.
- Tái sử dụng.
- Hỗ trợ triển khai nhanh chóng.

2.4. Bảo mật hệ thống

2.4.1. Mô hình CIA

Mô hình CIA đại diện cho ba nguyên tắc nền tảng cốt lõi của an toàn thông tin bao gồm: Tính bảo mật (Confidentiality), Tính toàn vẹn (Integrity) và Tính sẵn sàng (Availability).

- Tính bảo mật đảm bảo rằng các thông tin nhạy cảm của hệ thống chỉ được phép truy cập bởi các cá nhân hoặc dịch vụ đã được cấp quyền hợp lệ.
- Tính toàn vẹn hướng tới việc bảo vệ dữ liệu khỏi các hành vi chỉnh sửa, giả mạo hoặc xóa bỏ trái phép từ các tác nhân độc hại.
- Tính sẵn sàng đảm bảo rằng hệ thống máy chủ và các dịch vụ ứng dụng luôn duy trì trạng thái hoạt động ổn định, sẵn sàng đáp ứng nhu cầu truy cập của người dùng hợp pháp bất cứ khi nào cần thiết.

Mọi giải pháp bảo mật áp dụng trong đề tài đều hướng tới việc củng cố vững chắc ba trụ cột này.

2.4.2. Nguyên tắc Least Privilege

Nguyên tắc đặc quyền tối thiểu (Least Privilege) quy định rằng một tài khoản người dùng, một tiến trình chương trình hoặc một dịch vụ hệ thống chỉ được phép cấp đúng các quyền hạn tối thiểu cần thiết để hoàn thành chức năng công việc được giao, tuyệt đối không cấp dư thừa bất kỳ đặc quyền nào khác. Trong thiết kế hệ thống, nguyên tắc này được hiện thực hóa bằng các quy tắc nghiêm ngặt: không sử dụng lệnh cấp quyền bừa bãi như `chmod 777` cho thư mục mã nguồn, không vận hành container bằng tài khoản root mặc định và không cấp quyền quản trị tối cao trên môi trường Cloud khi chưa có lý do cụ thể. Điều này giúp thu hẹp tối đa phạm vi ảnh hưởng nếu một thành phần trong hệ thống bị xâm nhập.

2.4.3. Defense in Depth

Defense in Depth là chiến lược bảo vệ nhiều lớp.

Ví dụ:

- Security Group.
- Docker Network.
- Nginx.
- Laravel Middleware.
- PostgreSQL Authentication.

Nếu một lớp bảo vệ bị vượt qua, các lớp còn lại vẫn tiếp tục bảo vệ hệ thống.

2.4.4. Security Misconfiguration

Security Misconfiguration là lỗi cấu hình sai hệ thống.

Theo OWASP Top 10 2021, đây là một trong những nguyên nhân hàng đầu gây mất an toàn thông tin.

Các ví dụ phổ biến:

- `chmod 777`.
- Lộ file `.env`.
- Expose database port.
- Chạy container với quyền root.

CHƯƠNG 3. XÂY DỰNG HỆ THỐNG

3.1. Mô hình hệ thống

Hệ thống được thiết kế và xây dựng dựa trên kiến trúc ba tầng độc lập bao gồm Web Server, Application Server và Database Server. Nhằm đảm bảo tính cô lập, dễ bảo trì và thuận tiện cho công tác cấu hình an toàn, mỗi thành phần chức năng sẽ được đóng gói riêng biệt bên trong một container độc lập.

Sơ đồ mô tả luồng tương tác logic của hệ thống được tổ chức như sau:

```
Người dùng -> Nginx Container -> Laravel Application Container (PHP-FPM)
-> PostgreSQL Database Container
```

Trong mô hình trên:

- Người dùng gửi yêu cầu HTTP đến Nginx.
- Nginx tiếp nhận và chuyển tiếp các yêu cầu động đến PHP-FPM.
- Laravel xử lý nghiệp vụ và truy xuất dữ liệu.
- PostgreSQL chịu trách nhiệm lưu trữ dữ liệu của ứng dụng.

Việc tách riêng từng thành phần giúp tăng khả năng mở rộng, bảo trì và nâng cao mức độ an toàn cho hệ thống.

3.2. Chuẩn bị môi trường

Để xây dựng hệ thống, các thành phần sau được sử dụng:

Thành phần	Phiên bản
Docker Desktop	28.x
Docker Compose	2.x
Laravel	12.x
PHP-FPM	8.5
Nginx	Latest
PostgreSQL	16
Hệ điều hành	Windows 11

Sau khi cài đặt Docker Desktop, tiến hành kiểm tra trạng thái hoạt động bằng các lệnh:

```
docker --version
docker compose version
```

```
admin@LAPTOP-V9LUB000 MINGW64 ~
$ docker --version
Docker version 27.3.1, build ce12230

admin@LAPTOP-V9LUB000 MINGW64 ~
$ docker compose version
Docker Compose version v2.30.3-desktop.1
```

Kết quả hiển thị phiên bản Docker và Docker Compose chứng tỏ môi trường đã được cài đặt thành công.

Tiếp theo, kiểm tra Docker Engine đang hoạt động:

```
docker ps
```

```
admin@LAPTOP-V9LUB000 MINGW64 ~
$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Nếu hệ thống trả về danh sách container hoặc kết quả rỗng mà không xuất hiện lỗi thì Docker đã sẵn sàng sử dụng.

3.3. Dockerfile

Để triển khai Laravel trong môi trường container, cần xây dựng Docker Image chứa PHP-FPM và các thư viện cần thiết.

Dockerfile được sử dụng như sau:

```
1  FROM php:8.5-fpm
2
3  RUN apt-get update && apt-get install -y \
4      libpq-dev \
5      libzip-dev \
6      zip unzip git curl \
7      && docker-php-ext-install pdo pdo_pgsql zip \
8      && apt-get clean
9
10 COPY --from=composer:2 /usr/bin/composer /usr/bin/composer
11
12 WORKDIR /var/www/html
13
14 COPY composer.json composer.lock ./
15 RUN composer install --no-scripts --no-autoloader --prefer-dist
16
17 COPY . .
18
19 RUN composer dump-autoload --optimize
20
21 RUN chown -R www-data:www-data storage bootstrap/cache \
22     && chmod -R 775 storage bootstrap/cache
23
24 EXPOSE 9000
25 CMD ["php-fpm"]
```

Dockerfile thực hiện các nhiệm vụ:

- Sử dụng PHP-FPM làm môi trường thực thi.
- Cài đặt thư viện PostgreSQL.
- Cài đặt Composer.
- Sao chép mã nguồn Laravel vào container.
- Cài đặt các package phụ thuộc.
- Khởi động PHP-FPM khi container chạy.

Sau khi build thành công, Docker Image sẽ chứa đầy đủ môi trường cần thiết để vận hành ứng dụng Laravel.

3.4. Docker Compose

Docker Compose được sử dụng để quản lý đồng thời nhiều container.

Tập cấu hình:

```
1  services:
2
3  app:
4    build:
5      context: .
6      dockerfile: Dockerfile
7    container_name: gift_app
8    restart: unless-stopped
9    working_dir: /var/www/html
10   environment:
11     - APP_ENV=local
12     - APP_DEBUG=true
13   volumes:
14     - ./var/www/html
15     - /var/www/html/vendor
16   depends_on:
17     - postgres
18   networks:
19     - frontend_net
20     - backend_net
21   deploy:
22     resources:
23       limits:
24         cpus: "0.5"
25         memory: 256M
26
```

```

26
27  ✓ nginx:
28      image: nginx:alpine
29      container_name: gift_nginx
30      restart: unless-stopped
31  ✓ ports:
32      - "8080:80"
33  ✓ volumes:
34      - ./var/www/html
35      - ./nginx/default.conf:/etc/nginx/conf.d/default.conf
36  ✓ depends_on:
37      - app
38  ✓ networks:
39      - frontend_net
40  ✓ deploy:
41  ✓     resources:
42  ✓         limits:
43             cpus: "0.25"
44             memory: 64M
45
46  postgres:
47      image: postgres:16-alpine
48      container_name: gift_shop_db
49      restart: unless-stopped
50      environment:
51          POSTGRES_DB: gift_shop
52          POSTGRES_USER: gift_shop_user
53          POSTGRES_PASSWORD: secret123
54      volumes:
55          - postgres_data:/var/lib/postgresql/data
56      networks:
57          - backend_net
58      deploy:
59          resources:
60              limits:
61                  cpus: "0.5"
62                  memory: 256M
63
64  volumes:
65      postgres_data:
66
67  networks:
68      frontend_net:
69      backend_net:

```

Docker Compose giúp khởi động toàn bộ hệ thống chỉ bằng một lệnh duy nhất.

3.5. Cấu hình Nginx

Nginx được sử dụng làm Reverse Proxy phía trước Laravel.

Tập cấu hình:

```
1  server {
2      listen 80;
3      server_name localhost;
4      root /var/www/html/public;
5      index index.php index.html;
6
7      location / {
8          try_files $uri $uri/ /index.php?$query_string;
9      }
10
11     location ~ /\.php$ {
12         fastcgi_pass app:9000;
13         fastcgi_index index.php;
14         include fastcgi_params;
15         fastcgi_param SCRIPT_FILENAME $realpath_root$fastcgi_script_name;
16         fastcgi_param DOCUMENT_ROOT $realpath_root;
17     }
18
19     location ~ /\.(!well-known).* {
20         deny all;
21     }
22 }
```

Cấu hình trên cho phép:

- Phục vụ nội dung web thông qua thư mục public.
- Chuyển các request PHP sang PHP-FPM.
- Hỗ trợ cơ chế routing của Laravel.

Nhờ sử dụng Reverse Proxy, người dùng không truy cập trực tiếp vào PHP-FPM mà chỉ giao tiếp với Nginx.

3.6. Triển khai và kiểm thử

Sau khi hoàn tất Dockerfile và Docker Compose, tiến hành build hệ thống:

```
docker compose up -d --build
```

Kiểm tra trạng thái container:

```
docker ps
```

Tất cả container phải ở trạng thái: `Up`

Nếu một container bị lỗi, sử dụng:

```
docker logs <container_name>
```

để kiểm tra nguyên nhân.

CHƯƠNG 4. PHÂN TÍCH RỦI RO TỪ CÁC LỖ HỒNG CẤU HÌNH

4.1. Giới thiệu chương

Sau khi hoàn thành quá trình triển khai Web Server bằng Docker, nhóm tiến hành đánh giá các rủi ro bảo mật từ những cấu hình chưa an toàn. Mục tiêu là phân tích mức độ ảnh hưởng đối với CIA Triad (Confidentiality, Integrity, Availability) mà không thực hiện các hoạt động tấn công mạng thực tế. Theo OWASP Top 10 (2025), Security Misconfiguration là nguyên nhân phổ biến dẫn đến rò rỉ dữ liệu. Do đó, việc nhận diện và đánh giá các cấu hình chưa an toàn là bước cần thiết trước khi áp dụng các biện pháp hardening.

4.2. Rủi ro từ việc công khai PostgreSQL ra mạng bên ngoài

4.2.1. Mô tả hiện trạng cấu hình

Trong môi trường phát triển, người quản trị thường mở công khai cổng 5432 (PostgreSQL mặc định) bằng cách khai báo ports: ["5432:5432"] trong Docker Compose, cho phép bất kỳ máy nào trên Internet kết nối trực tiếp tới cơ sở dữ liệu, bypass hoàn toàn Nginx và Laravel.

4.2.2. Nguyên nhân phát sinh

Nguyên nhân chủ yếu xuất phát từ nhu cầu thuận tiện trong quá trình phát triển và kiểm thử. Nhà phát triển thường cần truy cập trực tiếp vào cơ sở dữ liệu để kiểm tra dữ liệu hoặc xử lý lỗi. Tuy nhiên, khi triển khai thực tế, cấu hình này không được loại bỏ, dẫn đến việc dịch vụ cơ sở dữ liệu bị lộ ra Internet.

4.2.3. Phân tích tác động theo CIA Triad

Việc công khai cổng PostgreSQL ra Internet tác động trực tiếp đến cả ba thành phần của mô hình CIA Triad. Dưới đây là phân tích chi tiết từng khía cạnh:

Tính bảo mật (Confidentiality): Khi cơ sở dữ liệu được công khai, mọi thông tin nhạy cảm được lưu trữ đều có nguy cơ bị truy cập trái phép. Các dữ liệu như tài khoản người dùng, mật khẩu hash, thông tin cá nhân, dữ liệu đơn hàng, lịch sử giao dịch, và các thông tin kinh doanh nhạy cảm khác có thể bị đánh cắp hoàn toàn nếu kẻ tấn công chiếm được quyền truy cập cơ sở dữ liệu. Rủi ro này càng lớn hơn nếu

như người quản trị sử dụng mật khẩu yếu hoặc mật khẩu mặc định cho tài khoản PostgreSQL.

Tính toàn vẹn (Integrity): Một khi đã chiếm được quyền truy cập cơ sở dữ liệu với tài khoản có quyền cao, kẻ tấn công có thể thực hiện các thao tác chỉnh sửa hoặc xóa dữ liệu. Điều này có thể dẫn đến việc thay đổi giá sản phẩm, xóa đơn hàng, chỉnh sửa lịch sử giao dịch, hoặc tạo các bản ghi giả mạo. Tác động của việc thay đổi dữ liệu có thể không chỉ gây hệ thống hoạt động sai lệch mà còn dẫn đến những quyết định kinh doanh sai lầm dựa trên dữ liệu bị thay đổi.

Tính sẵn sàng (Availability): Kẻ tấn công có thể thực hiện các truy vấn nặng, xóa toàn bộ dữ liệu trong bảng, hoặc thực hiện tấn công từ chối dịch vụ (Denial of Service) để làm cho cơ sở dữ liệu quá tải và không phản hồi các yêu cầu hợp lệ từ ứng dụng. Kết quả là hệ thống sẽ ngừng hoạt động, ảnh hưởng trực tiếp đến trải nghiệm của người dùng cuối.

4.2.4. Liên hệ OWASP Security Misconfiguration

Theo OWASP Top 10 2025, Security Misconfiguration xếp hạng A05, là một trong những lỗ hổng phổ biến và nguy hiểm nhất trong các ứng dụng web. Lỗi công khai cổng PostgreSQL ra Internet là một ví dụ điển hình của loại lỗ hổng này. OWASP xác định rằng Security Misconfiguration bao gồm việc: không áp dụng các bản vá bảo mật cần thiết, giữ lại tính năng không cần thiết, sử dụng tài khoản và mật khẩu mặc định, công khai các cổng dịch vụ nhạy cảm, và thiếu sự siết chặt cấu hình bảo mật. Lỗi công khai PostgreSQL phù hợp hoàn toàn với định nghĩa này, vì nó là kết quả của việc không áp dụng nguyên tắc "Default Deny" trong cấu hình tường lửa (Security Group).

4.2.5. Nhận xét

Lỗi công khai PostgreSQL ra Internet là một sai sót cấu hình nghiêm trọng với mức độ rủi ro cao (CVSS 8.8). Mặc dù về mặt kỹ thuật là một lỗi nhỏ, nhưng nó có thể dẫn đến những hậu quả thảm khốc cho hệ thống: mất mát dữ liệu toàn bộ, tiết lộ thông tin nhạy cảm, và gián đoạn dịch vụ. Nguyên nhân chính xuất phát từ sự hiểu lầm giữa phát triển và sản xuất, cũng như thiếu sự giáo dục về bảo mật trong đội ngũ phát triển. Biện pháp khắc phục là bắt buộc và cần được thực hiện ngay lập tức trong môi trường sản xuất.

4.3. Rủi ro từ Docker Socket Exposure

4.3.1. Vai trò của Docker Socket trong hệ thống

Docker Engine hoạt động dựa trên kiến trúc client-server, trong đó Docker Daemon (server) quản lý toàn bộ các container, image, network và volume. Giao tiếp giữa Docker CLI (client) và Docker Daemon được thực hiện thông qua một Unix Domain Socket có đường dẫn mặc định là `/var/run/docker.sock` trên các hệ thống Linux. Tập socket này đóng vai trò như một điểm truy cập duy nhất để kiểm soát toàn bộ Docker Engine. Bất kỳ tiến trình hoặc ứng dụng nào có quyền đọc/ghi vào socket này đều có khả năng thực hiện toàn bộ các thao tác mà Docker Daemon cho phép, bao gồm: tạo container mới, xóa container hiện có, truy cập tệp tin trên host, và thậm chí chiếm quyền kiểm soát toàn bộ hệ điều hành máy chủ.

4.3.2. Nguyên nhân phát sinh

Lỗi mount Docker Socket vào container thường xuất phát từ những nhu cầu thực tế nhất định. Các nhà phát triển hoặc quản trị viên có thể cần Docker Socket để: xây dựng Docker Images bên trong container (Docker-in-Docker), thực hiện continuous integration và deployment, quản lý container từ bên trong ứng dụng, hoặc thực hiện các tác vụ quản trị đặc biệt. Tuy nhiên, do thiếu nhận thức về các rủi ro bảo mật đi kèm, họ thường mount socket này mà không cài đặt bất kỳ cơ chế kiểm soát truy cập hoặc giới hạn quyền nào. Điều này tương đương với việc cấp toàn bộ quyền quản trị Docker cho ứng dụng bên trong container, có thể dẫn đến những hậu quả không lường trước nếu ứng dụng đó bị xâm phạm.

4.3.3. Tác động đến khả năng cô lập container

Khả năng cô lập là một trong những tính chất cốt lõi của công nghệ container. Container được thiết kế để tạo một môi trường độc lập, trong đó ứng dụng chạy được cách ly khỏi các container khác và từ hệ thống máy chủ. Tuy nhiên, khi Docker Socket được mount vào container, sự cô lập này bị phá vỡ hoàn toàn. Container có thể:

1. Tạo container mới trên cùng Docker Host, có khả năng bypass các cơ chế cô lập
2. Truy cập tệp tin trên máy chủ thông qua việc tạo container với volume được mount

3. Quét các container khác và truy cập vào dữ liệu của chúng
4. Tương tác với Docker Network để thực hiện tấn công mạng nội bộ
5. Thay đổi cấu hình Docker Host

Kết quả là, một ứng dụng bên trong container không còn được cô lập mà thực chất trở thành một phần không thể tách rời của hệ thống máy chủ, có khả năng tác động toàn diện đến toàn bộ cơ sở hạ tầng.

4.3.4. Phân tích tác động theo CIA Triad

Docker Socket Exposure tác động đến cả ba thành phần của CIA Triad, và mức độ tác động thậm chí còn lớn hơn so với PostgreSQL Exposure:

Tính bảo mật (Confidentiality): Kẻ tấn công có thể tạo một container mới được mount với thư mục root của host, từ đó đọc được toàn bộ tệp tin trên máy chủ, bao gồm các file .env chứa credentials, SSH keys, database passwords, API keys, và các dữ liệu nhạy cảm khác. Thêm vào đó, có thể truy cập vào các volume của PostgreSQL để trích xuất dữ liệu trực tiếp từ cơ sở dữ liệu.

Tính toàn vẹn (Integrity): Kẻ tấn công có khả năng chỉnh sửa bất kỳ tệp tin nào trên máy chủ host, bao gồm chính mã nguồn ứng dụng, cấu hình hệ điều hành, cấu hình Docker, hay thậm chí các tệp tin hệ thống quan trọng khác.

Tính sẵn sàng (Availability): Kẻ tấn công có thể xóa toàn bộ container, image, volume, hay thậm chí tắt Docker Engine, dẫn đến gián đoạn hoàn toàn của dịch vụ.

4.3.5. Liên hệ CIS Docker Benchmark

CIS Docker Benchmark là một bộ hướng dẫn chi tiết được phát triển bởi Center for Internet Security nhằm giúp các tổ chức cấu hình Docker một cách an toàn. Một trong những khuyến nghị quan trọng nhất của CIS Docker Benchmark là: không nên mount Docker Socket vào container, ngoại trừ những trường hợp cực kỳ cần thiết và có biện pháp kiểm soát rất nghiêm ngặt. Cụ thể, CIS đưa ra khuyến nghị:

1. Giới hạn quyền truy cập tới Docker Socket trên host
2. Không mount docker.sock vào container sản xuất
3. Nếu cần mount, hãy áp dụng các cơ chế xác thực và phân quyền chi tiết
4. Sử dụng các giải pháp thay thế như Docker API hoặc các công cụ chuyên dụng

Lỗi mount Docker Socket mà không có bất kỳ cơ chế kiểm soát nào hoàn toàn vi phạm các khuyến nghị này.

4.3.6. Nhận xét

Docker Socket Exposure là một lỗ hổng nguy hiểm nhất trong các lỗi cấu hình được phân tích. Với điểm CVSS 9.8 (Critical) và khả năng tác động vượt quá phạm vi container (Scope: Changed), lỗi này có thể dẫn đến hoàn toàn mất kiểm soát hệ thống. Kẻ tấn công có thể chiếm quyền kiểm soát toàn bộ Docker Host, trích xuất toàn bộ dữ liệu nhạy cảm, và thậm chí sử dụng máy chủ này làm bàn đạp để tấn công các hệ thống khác trong cơ sở hạ tầng. Biện pháp khắc phục là loại bỏ hoàn toàn Docker Socket trừ khi có những lý do cực kỳ chính đáng, và trong những trường hợp đó cũng phải áp dụng các cơ chế kiểm soát rất chặt chẽ.

4.4. Rủi ro từ Container chạy quyền Root và Flat Network

4.4.1. Đặc điểm quyền Root trong Docker

Theo cơ chế mặc định, khi một container được khởi động từ một image mà không khai báo người dùng cụ thể, toàn bộ các tiến trình bên trong container sẽ chạy dưới quyền của tài khoản root (uid=0). Mặc dù quyền root bên trong container bị hạn chế hơn so với quyền root trên host nhân virtualized tại Kernel cấp (do các Linux Capability được áp dụng mặc định), sự khác biệt này có thể bị phá vỡ hoặc bypass trong nhiều trường hợp. Nếu ứng dụng bên trong container bị khai thác một lỗ hổng, kẻ tấn công sẽ có quyền root và có khả năng thực hiện các hành động như: sửa đổi cấu hình hệ thống, cài đặt công cụ tấn công, thay đổi mã nguồn ứng dụng, hoặc kết hợp với các lỗi cấu hình khác để leo thang đặc quyền và chiếm quyền kiểm soát host.

4.4.2. Đặc điểm của Flat Network

Flat Network là mô hình mạng Docker mặc định khi sử dụng Docker Compose mà không khai báo các mạng riêng biệt. Trong cấu hình này, tất cả container được tạo ra đều được kết nối tới cùng một mạng bridge (thường là "bridge" mặc định), cho phép chúng liên lạc trực tiếp với nhau mà không cần bất kỳ hạn chế nào. Điều này có nghĩa là:

1. Nginx container có thể kết nối trực tiếp tới PostgreSQL container
2. Bất kỳ container nào cũng có thể quét các container khác
3. Không có sự phân tách giữa các tầng ứng dụng
4. Nếu một container bị xâm phạm, kẻ tấn công có thể truy cập trực tiếp tới tất cả dịch vụ khác

Mặc dù Flat Network đơn giản và thuận tiện cho phát triển, nhưng nó hoàn toàn không thích hợp cho môi trường sản xuất vì nó violate nguyên tắc Defense in Depth.

4.4.3. Phân tích nguy cơ mở rộng phạm vi ảnh hưởng

Sự kết hợp giữa container chạy quyền root và flat network tạo ra một con đường tấn công rất nguy hiểm gọi là "lateral movement". Quy trình này diễn ra như sau:

Bước 1 - Khai thác ứng dụng: Kẻ tấn công khai thác một lỗ hổng trong ứng dụng Laravel (ví dụ: RCE thông qua upload file). Vì ứng dụng chạy dưới quyền root, kẻ tấn công cũng có quyền root bên trong container.

Bước 2 - Quét mạng: Kẻ tấn công sử dụng các công cụ như nmap hoặc netstat để quét mạng flat và phát hiện các container khác đang chạy trên cùng mạng, bao gồm PostgreSQL (port 5432).

Bước 3 - Tấn công các dịch vụ khác: Kẻ tấn công thực hiện các cuộc tấn công brute force hoặc khai thác các lỗ hổng trong PostgreSQL để chiếm quyền truy cập.

Bước 4 - Trích xuất dữ liệu: Sau khi chiếm quyền PostgreSQL, toàn bộ dữ liệu trong cơ sở dữ liệu bị lộ.

Việc sử dụng quyền root giúp cho bước 2 trở nên dễ dàng hơn, vì các công cụ quét mạng thường cần quyền elevated. Nếu container chạy dưới quyền của một user thông thường, quá trình quét sẽ gặp nhiều hạn chế.

4.4.4. Đánh giá tác động theo CIA Triad

Tính bảo mật: Một khi ứng dụng bị xâm phạm, kẻ tấn công có quyền root và flat network cho phép tiếp cận tất cả dịch vụ khác, dẫn đến rủi ro lộ dữ liệu toàn bộ hệ thống.

Tính toàn vẹn: Kẻ tấn công có thể sửa đổi mã nguồn ứng dụng, thay đổi dữ liệu cơ sở dữ liệu, hoặc chèn mã độc.

Tính sẵn sàng: Kẻ tấn công có thể làm gián đoạn dịch vụ bằng cách tiêu tốn tài nguyên, xóa dữ liệu, hoặc tấn công các dịch vụ khác.

4.4.5. Nhận xét

Sự kết hợp giữa container chạy quyền root và flat network tạo ra một môi trường thuận lợi cho các cuộc tấn công lateral movement. Tuy điểm CVSS 8.6 (High) thấp hơn Docker Socket Exposure, tác động thực tế của lỗi này không nên bị xem nhẹ. Nó thể hiện sự thiếu hiểu biết về các nguyên tắc bảo mật cơ bản như Least Privilege và Defense in Depth. Biện pháp khắc phục bao gồm: chuyển sang non-root user, phân tách mạng Docker thành frontend và backend, và áp dụng các cơ chế kiểm soát truy cập mạng chi tiết.

CHƯƠNG 5. GIẢI PHÁP KHẮC PHỤC VÀ HARDENING HỆ THỐNG

5.1. Giới thiệu chương

Chương này trình bày chi tiết các biện pháp khắc phục kỹ thuật cụ thể nhằm triệt hạ hoàn toàn các lỗ hổng cấu hình hệ thống đã được nhận diện và phân tích ở giai đoạn trước. Mục tiêu cốt lõi của tiến trình không chỉ dừng lại ở việc xóa bỏ các nguy cơ an ninh tức thời, mà còn hướng tới việc thiết lập một kiến trúc an toàn thông tin bền vững dựa trên nguyên lý phòng thủ theo chiều sâu (Defense in Depth). Các giải pháp thiết kế cấu hình và đóng gói rào cản được xây dựng nghiêm ngặt dựa trên việc tuân thủ các khuyến nghị của bộ tiêu chuẩn CIS Docker Benchmark và danh mục kiểm soát cấu hình sai OWASP Top 10.

5.2. Khắc phục PostgreSQL Exposure

5.2.1. Thiết kế lại mô hình truy cập Database

Nhằm loại bỏ hoàn toàn bề mặt tấn công hướng ra Internet, mô hình kết nối tầng dữ liệu được tái cấu trúc theo nguyên lý cô lập tuyệt đối: chỉ cho phép container chứa ứng dụng (Laravel) thiết lập kết nối nội bộ tới container cơ sở dữ liệu (PostgreSQL). Hệ thống thực hiện cấm hoàn toàn mọi nỗ lực định tuyến hoặc ánh xạ cổng dịch vụ trực tiếp từ môi trường mạng diện rộng bên ngoài. Giải pháp này được hiện thực hóa đồng bộ bằng ba lớp phòng thủ: phân đoạn mạng nội bộ Docker (Docker Internal Network), thiết lập chính sách tường lửa máy chủ Linux (UFW - Uncomplicated Firewall) theo nguyên tắc Default Deny, và ứng dụng cơ chế đường hầm mã hóa mã nguồn mở (SSH Tunneling) khi có nhu cầu quản trị từ xa.

5.2.2. Triển khai phân đoạn mạng cô lập nội bộ (Docker Internal Network)

Hệ thống tiến hành hủy bỏ kiến trúc mạng phẳng mặc định (Flat Network) và khởi tạo một phân đoạn mạng ảo độc lập có tính chất cô lập cao mang tên `backend_net` bằng cách sử dụng trình điều khiển kết nối bridge được cấu hình riêng biệt trong Docker Compose:

```
networks:
  backend_net:
    driver: bridge
    internal: true # Chỉ định phân đoạn mạng hoàn toàn nội bộ, cách ly
với bên ngoài
```

Cơ chế này đảm bảo rằng: container PostgreSQL chỉ nhìn thấy container Laravel trong cùng mạng, không có container nào ngoài mạng này có thể kết nối, và tuyệt đối không có truy cập từ Internet. Docker sẽ tự động quản lý DNS resolution giữa các container dựa trên tên dịch vụ, cho phép container Laravel kết nối tới PostgreSQL bằng hostname "db" thay vì localhost hoặc IP address.

5.2.3. Cấu hình tường lửa máy chủ (UFW) theo nguyên tắc Default Deny

Tường lửa cục bộ của hệ điều hành Linux (UFW) được thiết lập theo triết lý bảo mật mặc định từ chối tất cả (Default Deny), chủ động khóa toàn bộ các cổng kết nối và chỉ mở quyền truy cập cho các dịch vụ được chỉ định tường minh. Kịch bản thiết lập quy chuẩn cho máy chủ Production được thực thi như sau:

```
# Thiết lập chính sách mặc định từ chối toàn bộ lưu lượng vào
sudo ufw default deny incoming

sudo ufw default allow outgoing

# Chỉ cho phép các dịch vụ công cộng thiết yếu
sudo ufw allow 80/tcp comment 'Allow HTTP'

sudo ufw allow 443/tcp comment 'Allow HTTPS'

# Chỉ cho phép SSH từ địa chỉ IP tĩnh của người quản trị để ngăn chặn
brute-force

sudo ufw allow from 203.0.113.50 to any port 22 proto tcp comment 'Admin
SSH'

# Kích hoạt hệ thống tường lửa
sudo ufw enable
```

Cấu hình này tạo ra một bộ lọc vòng ngoài kiên cố. Ngay cả khi có sự sai sót trong lệnh runtime của Docker, tường lửa UFW của hệ điều hành Host sẽ chủ động DROP (loại bỏ) toàn bộ các gói tin hướng tới cổng 5432 hoặc cổng 9000 (PHP-FPM) phát sinh từ môi trường Internet.

5.2.4. Đánh giá kết quả

Sau khi áp dụng giải pháp cô lập tầng mạng kết hợp tường lửa cục bộ, bề mặt rủi ro của dịch vụ PostgreSQL được giải trừ hoàn toàn. Cổng dịch vụ 5432 chuyển dịch trạng thái về vùng hoàn toàn ẩn danh đối với môi trường bên ngoài. Điểm số định lượng nguy hại CVSS v3.1 giảm từ mức 9.8 (Critical) về mức Không áp dụng (N/A) do vector tấn công từ xa qua môi trường mạng diện rộng (AV: Network) đã bị triệt tiêu tận gốc.

5.3. Khắc phục Docker Socket Exposure

5.3.1. Loại bỏ Docker Socket khỏi Container

Biện pháp khắc phục hiệu quả nhất cho Docker Socket Exposure là hoàn toàn loại bỏ Docker Socket khỏi container. Nếu mount `/var/run/docker.sock` là một yêu cầu thực sự cần thiết (ví dụ: cho CI/CD hoặc công cụ quản lý container), cần phải có những lý do cực kỳ chính đáng và phải áp dụng các cơ chế kiểm soát rất chặt chẽ. Trong bối cảnh của hệ thống Web Server đơn giản này, mount Docker Socket là hoàn toàn không cần thiết.

Cấu hình trước:

```
volumes:
```

- `/var/run/docker.sock:/var/run/docker.sock`

Cấu hình sau (đúng):

```
volumes:
```

- `./storage:/var/www/html/storage`

Chỉ mount các volume thực sự cần thiết cho ứng dụng. Điều này loại bỏ hoàn toàn nguy cơ container escape và chiếm quyền Docker Host.

5.3.2. Phân quyền Docker trên Host

Ngoài việc loại bỏ Docker Socket khỏi container, cần kiểm soát chặt chẽ quyền truy cập Docker trên host. Docker là một tool cực kỳ mạnh mẽ, vì vậy chỉ những người quản trị đáng tin cậy mới được cấp quyền:

- Không thêm user thông thường vào nhóm Docker (docker group)
- Không chia sẻ SSH keys giữa các thành viên đội
- Sử dụng key-based authentication với passphrase mạnh
- Theo dõi và audit tất cả các thao tác Docker trên production

Nếu cần cho phép một số người quản trị sử dụng Docker, sử dụng sudo với logging chi tiết:

```
sudo docker ps
sudo docker logs <container>
```

Cách tiếp cận này tạo nên một dấu vết audit tốt và giúp phát hiện các hoạt động bất thường.

5.3.3. Tăng cường bảo vệ Docker Host

Để tăng cường bảo vệ Docker Host, áp dụng các biện pháp sau:

- Cập nhật Docker Engine thường xuyên: Luôn sử dụng phiên bản Docker mới nhất để đảm bảo tất cả các lỗ hổng đã được vá. Sử dụng công cụ như docker scout hoặc trivy để kiểm tra lỗ hổng trong images.

- Không chạy Docker container với flag --privileged: Flag này cấp toàn bộ quyền cho container, có thể dẫn đến chiếm quyền host. Ngoại trừ những trường hợp cực kỳ cần thiết, tuyệt đối tránh sử dụng.

- Áp dụng Linux Capabilities một cách chặt chẽ: Thay vì cấp toàn bộ quyền, chỉ cấp những capabilities thực sự cần thiết. Ví dụ: drop ALL capabilities và chỉ add những thứ cần thiết:

```
cap_drop:
- ALL
cap_add:
- NET_BIND_SERVICE
```

Kích hoạt AppArmor hoặc SELinux: Các cơ chế này cung cấp một tầng bảo vệ bổ sung bằng cách giới hạn các hành vi mà một tiến trình hoặc container có thể thực hiện.

5.3.4. Đánh giá kết quả

Sau khi loại bỏ Docker Socket, lỗ hổng Docker Socket Exposure được khắc phục hoàn toàn. Container không còn có quyền kiểm soát Docker Host, và nguy cơ container escape được loại bỏ. Mức độ rủi ro giảm từ CVSS 9.8 (Critical) xuống 0, vì lỗ hổng đã không còn tồn tại.

5.4. Áp dụng nguyên tắc Least Privilege

5.4.1. Chuyển sang Non-Root User

Nguyên tắc Least Privilege yêu cầu container không nên chạy dưới quyền root. Thay vào đó, cần tạo một user thông thường với các quyền hạn chế. Trong Dockerfile:

```
RUN addgroup --gid 1000 appgroup
RUN adduser --uid 1000 --gid 1000 --disabled-password --gecos "" appuser

USER appuser
```

Cấu hình này tạo một user "appuser" với uid 1000, sau đó tất cả các tiến trình trong container sẽ chạy dưới quyền của user này thay vì root. Nếu ứng dụng bị khai thác, kẻ tấn công chỉ có quyền của user appuser, không thể thực hiện các hành động cần quyền root.

5.4.2. Kiểm soát quyền truy cập tập tin

Quyền truy cập tập tin phải được đặt một cách cẩn thận để đảm bảo mỗi thành phần chỉ có quyền cần thiết:

Thư mục source code: `chmod 755` (owner có thể đọc/ghi/thực hiện, group và others chỉ đọc/thực hiện)

Thư mục storage (nơi Laravel lưu trữ): `chmod 775` (owner và group có thể đọc/ghi/thực hiện)

Tập tin nhạy cảm (.env): `chmod 600` (chỉ owner có thể đọc/ghi)

Cơ sở dữ liệu volume: `chmod 700` (chỉ owner có thể truy cập)

Tuyệt đối tránh sử dụng `chmod 777` trên bất kỳ tập tin hay thư mục nào, vì điều này cho phép mọi người trên hệ thống có thể sửa đổi.

5.4.3. Hạn chế đặc quyền không cần thiết

Các tính năng Docker cho phép giới hạn các đặc quyền không cần thiết:

Read-only filesystem: Nếu ứng dụng không cần ghi dữ liệu vào filesystem root, có thể đặt toàn bộ filesystem là read-only:

```
read_only: true
```


Sau đó chỉ mount các volume có thể ghi:

```
volumes:  
  - ./storage:/app/storage
```

No new privileges: Ngăn chặn container từ tăng đặc quyền thông qua SUID/SGID bits:

```
security_opt:  
  - no-new-privileges:true
```

Không cấp quyền net.admin: Ngăn chặn container từ thay đổi cấu hình mạng:

```
sysctls:  
  - net.ipv4.ip_forward=0
```

5.4.4. Đánh giá hiệu quả

Sau khi áp dụng Least Privilege, khả năng leo thang đặc quyền được giảm đáng kể. Ngay cả nếu ứng dụng bị khai thác, kẻ tấn công sẽ phải đối mặt với nhiều rào cản:

Quyền giới hạn của non-root user

Filesystem read-only không cho phép cài đặt công cụ tấn công

Linux Capabilities bị drop ngăn chặn các thao tác cần quyền cao

Các cơ chế như AppArmor hoặc SELinux tạo thêm lớp bảo vệ

Lỗ hổng Container Root + Flat Network được giảm mức độ rủi ro từ CVSS 8.6 xuống thấp hơn đáng kể.

5.5. Phân tách mạng Docker

5.5.1. Frontend Network

Frontend Network là mạng Docker được công khai ra Internet, chỉ chứa duy nhất container Nginx:

```

networks:
  frontend_net:
    driver: bridge

services:
  nginx:
    networks:
      - frontend_net

```

Container Nginx tiếp nhận các yêu cầu HTTP/HTTPS từ người dùng Internet. Đây là điểm duy nhất mà các yêu cầu bên ngoài có thể vào hệ thống.

5.5.2. Backend Network

Backend Network là mạng Docker riêng biệt không được công khai, chỉ chứa các dịch vụ nội bộ:

```

networks:
  backend_net:
    driver: bridge

services:
  app:
    networks:
      - backend_net
  db:
    networks:
      - backend_net

```

Container Laravel và PostgreSQL chạy trên mạng này, hoàn toàn cô lập khỏi Internet.

5.5.3. Kiểm soát luồng giao tiếp

Với cấu hình mạng tách biệt này, luồng giao tiếp được kiểm soát chặt chẽ:

```

Internet -> Nginx (Frontend Network) -> Laravel (Backend Network) ->
PostgreSQL (Backend Network)

```

Nginx được kết nối tới cả Frontend Network và Backend Network, cho phép nó chuyển tiếp yêu cầu từ Internet tới Laravel. Tuy nhiên, Laravel chỉ được kết nối tới

Backend Network, không thể tiếp nhận các yêu cầu trực tiếp từ Internet. PostgreSQL hoàn toàn được cô lập, chỉ có thể truy cập từ Laravel.

Cấu hình:

```
services:
  nginx:
    networks:
      - frontend_net
      - backend_net
  app:
    networks:
      - backend_net
  db:
    networks:
      - backend_net
```

5.5.4. Đánh giá hiệu quả

Phân tách mạng Docker giúp thực hiện nguyên tắc Defense in Depth bằng cách tạo ra một ranh giới bảo mật giữa các tầng ứng dụng. Ngay cả nếu Nginx bị xâm phạm, kẻ tấn công vẫn bị giới hạn trong Frontend Network. Để truy cập PostgreSQL, kẻ tấn công phải:

1. Vượt qua Nginx (Frontend Network)
2. Vượt qua các cơ chế bảo mật của Laravel
3. Vượt qua Security Group của Docker
4. Vượt qua xác thực PostgreSQL
5. Vượt qua các kiểm tra tiếp theo

Mỗi lớp này là một rào cản bổ sung, làm tăng độ khó của cuộc tấn công exponentially.

5.6. Hardening hệ thống

5.6.1. Bảo vệ file .env

Tệp .env chứa những thông tin cực kỳ nhạy cảm như database credentials, API keys, và secret keys. Cần bảo vệ nó bằng:

Nginx configuration - Chặn truy cập trực tiếp:

```
location ~ /\. {  
    deny all;  
    return 403;  
}
```

Cấu hình này ngăn chặn bất kỳ yêu cầu nào tới các tệp tin bắt đầu bằng dấu chấm (hidden files) bao gồm .env.

Permission - Giới hạn quyền đọc:

```
chmod 600 .env
```

Chỉ owner (user appuser trong container) có thể đọc tệp tin này.

Secrets Management - Sử dụng Docker Secrets:

Thay vì lưu trữ secrets trong .env, sử dụng các dịch vụ quản lý secrets chuyên dụng. Docker Secrets cho phép cấp secrets tới container mà không cần đặt chúng vào environment variables hoặc tệp tin.

5.6.2. Logging và Monitoring

Ghi log (Logging) là lớp phòng thủ cuối cùng, cho phép phát hiện các hành vi bất thường sau khi chúng xảy ra. Cần ghi log từ:

Nginx Access Logs: Ghi lại tất cả các yêu cầu HTTP, giúp phát hiện các cuộc tấn công pattern như SQLi, XSS, hoặc brute force.

Nginx Error Logs: Ghi lại các lỗi xảy ra trong Nginx, giúp phát hiện các vấn đề cấu hình.

Laravel Logs: Lưu trữ logs từ ứng dụng Laravel, bao gồm các lỗi, exceptions, và các sự kiện quan trọng.

Docker Logs: Ghi lại các thông báo từ Docker daemon, giúp phát hiện các vấn đề liên quan đến container.

PostgreSQL Logs: Ghi lại các truy vấn và sự kiện cơ sở dữ liệu, giúp phát hiện các cuộc tấn công hoặc truy cập bất thường.

Logs nên được ghi lại một cách tập trung (centralized logging) bằng ELK Stack hoặc các dịch vụ tương tự để dễ dàng phân tích và phát hiện các mẫu tấn công.

5.6.3. Cập nhật Docker Image

Docker Images phải được cập nhật thường xuyên để đảm bảo tất cả các lỗ hổng đã được vá:

Cập nhật base images: Chọn các base images được maintain tốt (ví dụ: ubuntu:24.04-lts, php:8.5-fpm) thay vì các images cũ hoặc không được maintain.

Kiểm tra lỗ hổng: Sử dụng các tool như docker scout, trivy, hoặc gypre để scan images:

```
docker scout quickview nginx:latest
```

```
trivy image postgres:17
```

Rebuild images định kỳ: Thậm chí nếu mã nguồn không thay đổi, cần rebuild images theo định kỳ để đưa vào các bản cập nhật từ base images.

Xóa images cũ: Xóa các images lỗi thời để giảm bề mặt tấn công.

5.6.4. Kiểm tra cấu hình định kỳ

Cấu hình bảo mật phải được kiểm tra định kỳ để đảm bảo không có sự thoái hóa:

Audit Security Group: Kiểm tra xem các công được công khai là đúng như kỳ vọng.

Audit Docker configuration: Kiểm tra các Dockerfiles, docker-compose.yml không có các cấu hình nguy hiểm.

Audit File Permissions: Kiểm tra các quyền tập tin không bị thay đổi.

Audit Network Configuration: Kiểm tra các mạng Docker vẫn được phân tách đúng cách.

Các audit này có thể được tự động hóa bằng các script hoặc các tool chuyên dụng.

5.7. Đánh giá sau khi khắc phục

5.7.1. So sánh trước và sau

Bảng dưới đây so sánh trạng thái bảo mật của hệ thống trước và sau khi áp dụng các biện pháp khắc phục:

Tiêu chí	Trước khắc phục	Sau khắc phục
PostgreSQL Exposure	Port 5432 công khai	Port 5432 đóng, Docker Network riêng
Docker Socket	Mount vào container	Loại bỏ hoàn toàn
User Privilege	Chạy quyền root	Non-root user (appuser)
Network Isolation	Flat Network	Frontend & Backend Networks
Security Group	Mở quá nhiều cổng	Default Deny, chỉ mở cần thiết
File Permissions	chmod 777 ở một số nơi	chmod 600-755 cụ thể
Logging	Không hoặc tối thiểu	Logging đầy đủ tất cả thành phần

5.7.2. Đánh giá theo CIA Triad

Sau khi áp dụng các biện pháp khắc phục, các yếu tố CIA Triad được cải thiện đáng kể:

Confidentiality (Bảo mật): Dữ liệu không còn bị tiếp xúc trực tiếp từ Internet. PostgreSQL được cô lập bằng Docker Network, file .env được bảo vệ bằng quyền hạn chế và Nginx configuration. Ngay cả nếu ứng dụng bị khai thác, kẻ tấn công gặp nhiều rào cản để truy cập dữ liệu.

Integrity (Toàn vẹn): Filesystem được đặt thành read-only, giảm khả năng kẻ tấn công chỉnh sửa các tệp tin quan trọng. Non-root user không có quyền thay đổi cấu hình hệ thống. Database access được kiểm soát qua xác thực.

Availability (Sẵn sàng): Mặc dù Availability chủ yếu phụ thuộc vào độ tin cậy của cơ sở hạ tầng chứ không phải bảo mật, các biện pháp khắc phục cũng giúp: Giảm bề mặt tấn công làm giảm nguy cơ DDoS, Logging giúp phát hiện và xử lý sự cố nhanh chóng.

5.7.3. Kết luận chương

Chương 5 đã trình bày các biện pháp khắc phục chi tiết cho từng lỗ hổng cấu hình được xác định trong Chương 4. Các giải pháp không chỉ xóa bỏ các rủi ro tức thời

mà còn thiết lập một kiến trúc bảo mật vững chắc dựa trên các nguyên tắc:

Least Privilege: Mỗi thành phần chỉ được cấp đúng các quyền cần thiết

Defense in Depth: Các lớp bảo vệ độc lập tạo nên sự phòng thủ toàn diện

Default Deny: Mặc định từ chối tất cả, chỉ cho phép những gì cần thiết

Logging & Monitoring: Theo dõi liên tục để phát hiện các hành vi bất thường

Sau khi áp dụng đầy đủ các biện pháp này, hệ thống được coi là đã đạt mức độ bảo mật phù hợp cho môi trường sản xuất nhỏ. Tuy nhiên, bảo mật là một quy trình liên tục, không bao giờ hoàn toàn. Cần tiếp tục cập nhật, kiểm tra định kỳ, và sẵn sàng áp dụng các biện pháp mới khi có những mối đe dọa mới xuất hiện.

CHƯƠNG 6. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

6.1. Kết quả đạt được

Sau quá trình nghiên cứu, xây dựng và triển khai hệ thống, đề tài đã hoàn thành các mục tiêu đã đề ra ban đầu. Hệ thống Web Server sử dụng Docker được triển khai thành công trên môi trường AWS EC2 với mô hình gồm Nginx, Laravel PHP-FPM và PostgreSQL.

Trong quá trình thực hiện, người thực hiện đã tìm hiểu và vận dụng được các công nghệ đang được sử dụng phổ biến trong thực tế như Docker, Docker Compose, Nginx Reverse Proxy và dịch vụ EC2 của AWS. Điều này giúp củng cố kiến thức về triển khai ứng dụng trong môi trường điện toán đám mây cũng như hiểu rõ hơn về quy trình vận hành của một hệ thống Web hiện đại.

Bên cạnh việc triển khai hệ thống, đề tài còn thực hiện mô phỏng các lỗ hổng cấu hình bảo mật thường gặp trong thực tế, bao gồm:

- Công khai cổng PostgreSQL ra Internet.
- Docker Socket Exposure.
- Container chạy bằng quyền Root.
- Thiết kế mạng Docker không được cô lập.

Thông qua các kịch bản mô phỏng, đề tài đã phân tích được các nguy cơ bảo mật có thể phát sinh, đánh giá mức độ ảnh hưởng đến hệ thống và liên hệ với các nguyên tắc bảo mật cơ bản như CIA Triad, Principle of Least Privilege và Defense in Depth.

Sau khi áp dụng các biện pháp hardening, hệ thống đã giảm đáng kể các điểm yếu bảo mật, nâng cao khả năng bảo vệ dữ liệu và hạn chế nguy cơ bị khai thác từ bên ngoài.

Nhìn chung, các mục tiêu nghiên cứu đặt ra trong đề tài đều đã được hoàn thành, đồng thời giúp người thực hiện có thêm kinh nghiệm thực tế trong việc triển khai và bảo vệ hệ thống trên môi trường điện toán đám mây.

6.2. Hạn chế của đề tài

Mặc dù đã đạt được các mục tiêu đề ra, đề tài vẫn còn một số hạn chế nhất định.

Thứ nhất, hệ thống được triển khai ở quy mô nhỏ nhằm phục vụ mục đích học tập và nghiên cứu. Do đó các vấn đề liên quan đến khả năng mở rộng, cân bằng tải và triển khai đa máy chủ chưa được xem xét.

Thứ hai, đề tài chỉ tập trung vào một số lỗi cấu hình phổ biến trong Docker và AWS EC2. Trong thực tế còn tồn tại nhiều nguy cơ khác như:

- Kubernetes Misconfiguration.
- Container Runtime Vulnerability.
- Supply Chain Attack.
- SSRF trên môi trường Cloud.
- IAM Privilege Escalation.

Những nội dung này chưa nằm trong phạm vi nghiên cứu của đề tài.

Thứ ba, hệ thống chưa triển khai các giải pháp giám sát chuyên sâu như Prometheus, Grafana hoặc ELK Stack. Do đó khả năng phát hiện và phản ứng trước các sự cố bảo mật vẫn còn hạn chế.

Ngoài ra, do điều kiện về thời gian và tài nguyên, đề tài chưa thực hiện các bài kiểm thử bảo mật chuyên sâu như Penetration Testing hoặc Vulnerability Assessment trên toàn bộ hệ thống.

6.3. Hướng phát triển

Trong tương lai, đề tài có thể được mở rộng theo nhiều hướng nhằm tiệm cận hơn với môi trường triển khai thực tế.

6.3.1. Triển khai HTTPS

Hệ thống hiện tại chủ yếu sử dụng giao thức HTTP. Trong môi trường thực tế cần triển khai HTTPS bằng Let's Encrypt nhằm:

- Mã hóa dữ liệu truyền tải.
- Bảo vệ thông tin người dùng.
- Ngăn chặn tấn công Man-in-the-Middle.

6.3.2. Xây dựng hệ thống giám sát

Tích hợp:

- Prometheus.
- Grafana.
- Loki.
- ELK Stack.

để theo dõi hiệu năng và phát hiện các dấu hiệu bất thường.

6.3.3. Triển khai CI/CD

Áp dụng GitHub Actions hoặc GitLab CI/CD để tự động hóa quá trình:

- Build Docker Image.
- Kiểm thử ứng dụng.
- Triển khai lên AWS.

Điều này giúp giảm sai sót trong quá trình vận hành.

6.3.4. Nghiên cứu Kubernetes

Docker Compose phù hợp với các hệ thống nhỏ, tuy nhiên trong môi trường doanh nghiệp thường sử dụng Kubernetes để:

- Quản lý container.
- Tự động mở rộng.
- Tự động phục hồi dịch vụ.

Đây là hướng nghiên cứu có giá trị thực tiễn cao trong lĩnh vực điện toán đám mây..

6.4. Bài học kinh nghiệm

Thông qua quá trình thực hiện đề tài, người thực hiện nhận thấy rằng việc triển khai thành công một ứng dụng Web mới chỉ là bước khởi đầu. Một hệ thống chỉ thực sự có giá trị khi đảm bảo được các yêu cầu về bảo mật, tính ổn định và khả năng vận hành lâu dài.

Một trong những bài học quan trọng nhất rút ra từ đề tài là nhiều sự cố bảo mật không xuất phát từ lỗi lập trình mà đến từ các sai sót trong cấu hình hệ thống. Chỉ

một thay đổi nhỏ như mở nhầm cổng cơ sở dữ liệu hoặc mount Docker Socket không đúng cách cũng có thể tạo điều kiện cho kẻ tấn công chiếm quyền kiểm soát toàn bộ hệ thống.

Đề tài cũng giúp người thực hiện hiểu rõ hơn về vai trò của các nguyên tắc bảo mật cơ bản như:

- Least Privilege.
- Defense in Depth.
- Secure by Default.

Đây là những nguyên tắc cần được áp dụng ngay từ giai đoạn thiết kế hệ thống thay vì chỉ bổ sung sau khi xảy ra sự cố.

Ngoài ra, việc triển khai thực tế trên AWS giúp người thực hiện tiếp cận gần hơn với môi trường làm việc của doanh nghiệp, hiểu được quy trình triển khai ứng dụng hiện đại và nâng cao kỹ năng quản trị hệ thống trong môi trường điện toán đám mây.

Có thể khẳng định rằng đề tài không chỉ mang lại kiến thức lý thuyết mà còn cung cấp nhiều kinh nghiệm thực hành có giá trị, góp phần chuẩn bị nền tảng cho quá trình học tập và làm việc trong lĩnh vực Công nghệ thông tin sau này.

TÀI LIỆU THAM KHẢO

[1]. **OWASP (Open Web Application Security Project).** *Docker Security Cheat Sheet*

https://cheatsheetseries.owasp.org/cheatsheets/Docker_Security_Cheat_Sheet.html

[2]. Docker Inc., "Docker Documentation - Get Started".

<https://docs.docker.com/>

[3]. The PHP Group, "PHP-FPM Official Documentation".

<https://www.php.net/manual/en/install.fpm.php>

[4]. PostgreSQL Global Development Group, "PostgreSQL Official Documentation"

<https://www.postgresql.org/docs/>

[5]. Laravel Foundation, "Laravel Documentation".

<https://laravel.com/docs/>